METAVUL: SUPPLEMENTARY MATERIALS

Ira Ceka[1], Arnav Garg[1], Ryan Lutz[2], Ting Zhang[3], Ratnadira Widyasari[4], Gail Kaiser[5], Baishakhi Ray[1]

[1]Columbia University [2]The Citadel [3]Monash University
[4]University of Victoria [5]Columbia University, Department of Computer Science

iceka@cs.columbia.edu ag5516@columbia.edu rlutz@student.citadel.edu
Ting.Zhang@monash.edu ratnadiraw@uvic.ca
kaiser@cs.columbia.edu rayb@cs.columbia.edu

Each section below is also available as a separate PDF at https://github.com/ARiSE-Lab/MetaVul/tree/main/supplementary_materials.

CONTENTS

S1 Validation Set Construction **2**

S2 Harness Quality and Trajectory Auditing **5**

S3 Seed Harness **7**

S4 Pairwise Accuracy Example **10**

S5 RQ1: Full Metric Breakdown **11**

S6 Additional Results **12**

S6.1 Expanded per-iteration plots 12

S6.2 Worked example: selecting the Pareto frontier 17

S7 GPT-4o Transfer Runs **18**

S8 Agent Sandboxing Details **19**

S8.1 Optimizer Agent 19

S8.2 Detection Agent (Evaluator) 20

S8.3 Workspace-Level Restrictions 20

S9 Run Configurations **21**

S10 Context Budget **22**

S11 Optimizer-Information Ablation **23**

S12 Hosted Inference Prices **24**

S1 VALIDATION SET CONSTRUCTION

TitanVul sampling. We draw the 150 TitanVul pairs from its 17,101 C/C++ pairs with a fixed random seed (42), sampling by CWE in proportion to each CWE's frequency:

- **Proportional share.** Each CWE receives a share of the 150 pairs in proportion to how often it appears in TitanVul C/C++.
- **Minimum of one.** Every CWE receives at least one pair, so rare CWEs are slightly overrepresented. If this pushes the total over budget, pairs are removed at random until the budget is met.
- **Unlabeled pairs.** Pairs without a usable CWE (a blank label, `NVD-CWE-noinfo` or `NVD-CWE-Other`) are sampled as their own group, also in proportion to their frequency, with a minimum of 5. This group contributes 84 pairs: 79 with a blank label and 5 with only an NVD placeholder.

CWE distribution. Table S1.1 lists every CWE in the validation set D and the held-out PrimeVul set.

- **TitanVul (150 pairs).** 79 pairs have no CWE label. The other 71 are labeled and cover 58 distinct CWEs; 26 of them carry several CWEs, such as `CWE-119,CWE-190,CWE-787`. Eight pairs include an NVD placeholder instead of, or alongside, a real CWE: `NVD-CWE-Other` (5) and `NVD-CWE-noinfo` (3). Five of these have no real CWE. The rest form a long tail: CWE-345, CWE-400 and CWE-787 have 3 pairs each, and most other CWEs appear once.
- **CWEval (52 pairs).** Every pair has exactly one CWE, covering 29 distinct CWEs.
- **Overall.** 123 of the 202 pairs have a CWE label, and D covers 72 distinct CWEs. CWEval leans toward web and cryptographic weaknesses, while TitanVul leans toward memory-safety bugs.
- **PrimeVul held-out set (430 pairs).** Every pair has exactly one CWE, covering 61 distinct CWEs, 31 of which also appear in D. Memory-safety bugs dominate: CWE-787 (70), CWE-125 (47), CWE-703 (47), CWE-476 (39) and CWE-416 (29) account for 232 of the 430 pairs, and 25 CWEs appear only once.

Table S1.1: All 102 CWEs in the validation set D (TitanVul and CWEval, 72 CWEs) and the held-out PrimeVul set (430 pairs, 61 CWEs), with the number of pairs from each source. 31 CWEs appear in both sets. A TitanVul pair with several CWEs counts once for each. Not listed: 79 TitanVul pairs with no CWE label, and the NVD placeholders `NVD-CWE-Other` (5 TitanVul pairs) and `NVD-CWE-noinfo` (3).

CWE	TitanVul	CWEval	PrimeVul
CWE-16	1	0	0
CWE-17	0	0	1
CWE-19	1	0	0
CWE-20	4	2	14
CWE-22	0	4	6
CWE-59	1	0	2
CWE-77	1	0	0
CWE-78	1	2	0
CWE-79	1	2	2
CWE-94	0	0	2
CWE-113	0	2	0
CWE-117	0	2	0
CWE-119	6	1	14

Continued on next page

Table S1.1 continued

CWE	TitanVul	CWEval	PrimeVul
CWE-120	4	1	7
CWE-121	1	0	0
CWE-122	2	0	3
CWE-125	6	1	47
CWE-129	1	0	0
CWE-131	0	1	0
CWE-134	1	1	1
CWE-170	0	1	0
CWE-189	1	0	5
CWE-190	2	1	11
CWE-191	1	0	2
CWE-193	0	0	2
CWE-200	1	0	16
CWE-212	1	0	1
CWE-252	0	0	1
CWE-255	1	0	0
CWE-264	0	0	1
CWE-269	1	0	4
CWE-275	0	0	1
CWE-276	1	0	2
CWE-284	0	0	4
CWE-287	1	0	1
CWE-288	0	0	1
CWE-295	2	0	1
CWE-310	0	0	2
CWE-311	1	0	0
CWE-320	1	0	0
CWE-326	0	4	0
CWE-327	1	4	1
CWE-329	0	2	0
CWE-331	1	0	0
CWE-345	3	0	0
CWE-346	1	0	0
CWE-347	2	2	0
CWE-354	0	0	2
CWE-362	5	0	8
CWE-369	1	0	14
CWE-377	0	1	0
CWE-388	1	0	0
CWE-391	1	0	0
CWE-399	0	0	3
CWE-400	3	1	3
CWE-401	0	0	8
CWE-415	0	0	9
CWE-416	1	0	29
CWE-434	0	0	1
CWE-436	1	0	0
CWE-444	0	0	1
CWE-459	1	0	0
CWE-468	0	1	0
CWE-476	4	1	39
CWE-494	1	0	0
CWE-522	0	0	1
CWE-552	0	0	1
CWE-601	1	0	0
CWE-617	0	0	12
CWE-643	0	2	0
CWE-665	0	0	1
CWE-667	1	0	0
CWE-668	2	0	1
CWE-672	1	0	1

Continued on next page

Table S1.1 continued

CWE	TitanVul	CWEval	PrimeVul
CWE-676	0	1	0
CWE-693	1	0	0
CWE-703	0	0	47
CWE-704	0	0	2
CWE-732	1	2	1
CWE-754	0	0	2
CWE-755	1	0	0
CWE-760	0	2	0
CWE-770	0	0	1
CWE-772	0	0	2
CWE-787	3	1	70
CWE-788	1	0	0
CWE-822	1	0	0
CWE-824	0	0	1
CWE-825	1	0	0
CWE-834	0	0	1
CWE-835	0	0	5
CWE-840	1	0	0
CWE-843	0	0	3
CWE-862	1	0	1
CWE-863	1	0	1
CWE-908	0	0	2
CWE-909	1	0	1
CWE-918	0	4	0
CWE-924	0	0	1
CWE-943	1	2	0
CWE-1284	1	0	0
CWE-1333	0	1	0

S2 HARNESS QUALITY AND TRAJECTORY AUDITING

We audit individual harnesses and their evolution across completed optimization iterations. The framework records both deterministic static analysis and LLM-based semantic judging separately, then tracks each result longitudinally.

Intrinsic harness evaluation. Static evaluation ingests the candidate harness and, only for token-efficiency comparison, a fixed seed harness. Semantic judging compares the candidate harness using a fixed rubric. Neither component reads code samples, labels, predictions, or downstream performance. The trajectory auditor separately reads the completed-iteration history and saved metrics for reporting. Thus, intrinsic evaluation determines whether a harness is structurally and semantically suitable for vulnerability analysis, rather than its efficacy in vulnerability identification. The semantic judge is prohibited from inferring accuracy, precision, recall, or vulnerability-detection performance.

For each harness h_i, the evaluator records four distinct values:

$$(Q_{\text{static}}, Q_{\text{semantic}}, E_{\text{static}}, E_{\text{semantic}})$$

Q_{static} and Q_{semantic} are quality scores, while E_{static} and E_{semantic} are eligibility decisions. The framework does not combine them into a composite score.

Quantitative static quality. Q_{static} ranges from 0 to 100 and comprises four components:

- Template integrity: 40 points
- Schema mechanics: 30 points
- Format hygiene: 15 points
- Seed-relative token efficiency: 15 points

Template integrity checks that the harness is valid text, contains exactly one code placeholder, and has no malformed or unknown template variables. Schema mechanics validates that the vulnerability-report structure is present and complete. Format hygiene checks duplicate instructions and headings, whitespace and line-ending consistency, and Markdown-fence balance. Token efficiency compares the candidate harness with the fixed seed harness.

Qualitative semantic quality. A modular LLM-as-judge evaluates the candidate harness against nine rubric dimensions: role alignment; task identity and primary task; input-object and untrusted-code boundary; SCoT Sequence, Branch, and Loop scaffold; scope and grounding; prohibited behavior; output-contract semantics; internal consistency and instruction precedence; and organization and maintainability. Each dimension is rated from 0–4. Ratings from valid judge attempts are averaged by dimension and weighted to produce Q_{semantic} on a 0–100 scale.

Eligibility criteria. Eligibility acts as a minimum validity gate distinct from quality scoring. E_{static} requires the essential template and schema conditions. E_{semantic} requires no critical semantic failure and average ratings of at least 2 out of 4 for the primary task, SCoT scaffold, scope/grounding, and output-contract dimensions. A harness can therefore receive a relatively high quality score yet remain ineligible.

Harness trajectory auditing. For every completed optimization iteration, the audit records static and semantic quality, eligibility, duplicates identified from normalized harness text, changed-line counts, optimizer change summaries, and saved downstream metrics. The latter include pairwise accuracy, accuracy, precision, recall, F1, parse failures, and sample count when available. The audit does not impose monotonic improvement or construct a combined trajectory score; each measurement remains separate so that changes across iterations can be examined directly.

Table S2.1 reports every validation Pareto-frontier harness (pairwise accuracy vs. joint hit rate). Ratings are the GPT-OSS-120B judge's 0–4 scores averaged over its valid runs; FP and Flagged compare the seed with each harness on the 404 validation functions.

Table S2.1: Every Pareto-frontier harness: validation metrics, judge ratings, and length.

Agent + Model	Iter.	Pairwise	Joint	SCoT	Prohib.	Output	Judge	Length	FP	Flagged
Codex + Inkling-Small	6	20.8	19.3	4	4	4	99.4	3.8×	185 → 123	378 → 262
	8	23.3	12.2	4	4	4	98.8	3.7×	185 → 117	378 → 256
	12	27.7	9.4	4	4	4	100.0	4.8×	185 → 104	378 → 243
Codex + DeepSeek V4 Flash	1	24.3	28.2	4	4	4	100.0	4.1×	160 → 70	341 → 178
	5	28.7	24.2	4	4	3	90.4	12.0×	160 → 89	341 → 224
	9	30.2	22.1	4	4	4	91.2	19.3×	160 → 64	341 → 175
	11	36.1	20.9	4	4	4	100.0	13.9×	160 → 65	341 → 191
	14	33.7	21.5	4	4	4	99.4	21.5×	160 → 72	341 → 191
Claude Code + Opus 4.6	2	31.2	31.3	4	4	4	100.0	7.8×	170 → 85	357 → 225
	3	33.7	28.6	4	4	4	100.0	7.7×	170 → 85	357 → 232
Claude Code + GLM-5.3	6	31.2	34.1	4	4	3.5	98.0	16.2×	134 → 69	297 → 192

Table S2.2 lists every harness that the judge rated below the top score on the prohibited-content or output-format dimension.

Table S2.2: Harnesses below the top rubric rating (GPT-OSS-120B judge, two runs per harness; a rating averages the valid runs). ‡ marks harnesses on the validation Pareto frontier; the only prohibited-content case is not on the frontier, and the two frontier cases concern output format only.

Harness	Dimension	Rating	Finding
Inkling-Small, iter 14	Prohibited content	3.0	Prefers "not vulnerable" when source or sink is unquoted
Inkling-Small, iter 13	Output format	3.5	SCoT reasoning required, then "only" a JSON object
DeepSeek V4 Flash, iter 5‡	Output format	3.0	SCoT reasoning required, then "only" a JSON object
GLM-5.3, iter 2	Output format	3.5	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers
GLM-5.3, iter 3	Output format	3.0	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers
GLM-5.3, iter 6‡	Output format	3.5	The example output schema leaves line numbers as `null`, but the instructions require integer line numbers

S3 SEED HARNESS

Each discovery run starts from the full SCoT seed harness below (iteration 0). The 151-line Python file prepares each function for analysis and validates the detector's JSON response. The optimizer modifies this file while preserving its overall structure.

```python
#!/usr/bin/env python3
"""
Standalone vulnerability-detection harness.

Plug this into any agent or model backend:
    from harness import build_prompt, parse_verdict
    prompt = build_prompt(code)          # send this to your model
    verdict = parse_verdict(raw_output)  # feed the model's raw text back in
"""
import json
import re
import sys
from typing import Any, Dict, Optional

PROMPT_TEMPLATE = 'You are an expert software engineer and security analyst.\n\nAnalyze the following code for security vulnerabilities.\n\nUse Structured Chain-of-Thought (SCoT): reason using explicit program structures:\n- Sequence (ordered steps)\n- Branch (if/else)\n- Loop (for/while)\n\nRules:\n- Base reasoning only on the given code.\n- Follow actual control flow (paths, checks, state changes).\n- Focus on security-relevant behavior (memory safety, bounds, lifetime, input validation, integer safety, authz/authn, unsafe APIs).\n- Do NOT rewrite or modify code.\n- After reasoning, output ONLY the JSON object described below.\n\nOutput format requirements:\n- First: write your SCoT reasoning in plain text with numbered steps.\n- Then: output ONLY one JSON object (no markdown, no extra text).\n\nRequired JSON schema:\n{\n  "verdict": {\n    "vulnerable": true/false,\n    "confidence": 0.0-1.0,\n    "cwe": ["CWE-..."],\n    "severity": "low|medium|high|critical|unknown",\n    "summary": "..."\n  },\n  "evidence": [\n    {\n      "file": null,\n      "function": null,\n      "location": {"start_line": null, "end_line": null},\n      "snippet": "...",\n      "reason": "..."\n    }\n  ],\n  "notes": {\n    "assumptions": [],\n    "limitations": []\n  }\n}\n\nCode:\n{{CODE}}\n'

REQUIRED_VERDICT_KEYS = {"vulnerable", "confidence", "cwe", "severity", "summary"}
REQUIRED_TOP_KEYS = {"verdict", "evidence", "notes"}


def build_prompt(code: str) -> str:
    """Render the harness prompt for a single code sample."""
    return PROMPT_TEMPLATE.replace("{{CODE}}", code)


def _strip_eos_tokens(raw: str) -> str:
    for tok in ("<|endoftext|>", "<|end|>", "<|im_end|>", "<|im_start|>"):
        raw = raw.replace(tok, "")
    return raw


def _sanitize_control_chars(raw: str) -> str:
    def _replace(m):
        c = m.group(0)
        return {'\t': '\\t', '\b': '\\b', '\f': '\\f'}.get(c, f'\\u{ord(c):04x}')
    return re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f]', _replace, raw)


def _find_verdict_object(text: str, strict: bool = True) -> Optional[Dict[str, Any]]:
    decoder = json.JSONDecoder(strict=strict)
    for m in re.finditer(r'\{', text):
        start = m.start()
        try:
            obj, _ = decoder.raw_decode(text, start)
        except json.JSONDecodeError:
            continue
        if not isinstance(obj, dict) or "verdict" not in obj:
            continue
        v = obj["verdict"]
        if isinstance(v, str) and "|" in v:
            continue
        if isinstance(v, dict) and isinstance(v.get("vulnerable"), bool):
            return obj
    return None


def _extract_verdict_direct(text: str) -> Optional[Dict[str, Any]]:
    for m in re.finditer(r'"verdict"\s*:\s*\{', text):
        vstart = m.end() - 1
```

```
        depth = 0
        for i in range(vstart, min(vstart + 5000, len(text))):
            if text[i] == '{':
                depth += 1
            elif text[i] == '}':
                depth -= 1
            if depth == 0:
                verdict_str = text[vstart:i + 1]
                for strict in (True, False):
                    try:
                        obj = json.loads(verdict_str, strict=strict)
                        if isinstance(obj, dict) and isinstance(obj.get("vulnerable"), bool):
                            return {"verdict": obj}
                    except json.JSONDecodeError:
                        continue
                break
    return None


def extract_json(raw: str) -> Optional[Dict[str, Any]]:
    """Find the first valid JSON verdict object in raw model output."""
    cleaned = _strip_eos_tokens(raw)

    result = _find_verdict_object(cleaned, strict=True)
    if result:
        return result

    result = _find_verdict_object(cleaned, strict=False)
    if result:
        return result

    sanitized = _sanitize_control_chars(cleaned)
    result = _find_verdict_object(sanitized, strict=True)
    if result:
        return result

    stripped = re.sub(r'```(?:json)?\s*', '', cleaned)
    result = _find_verdict_object(stripped, strict=False)
    if result:
        return result

    return _extract_verdict_direct(cleaned)


def validate_schema(obj: Dict[str, Any]) -> list:
    errors = []
    for k in REQUIRED_TOP_KEYS:
        if k not in obj:
            errors.append(f"missing top-level key: {k}")
    verdict = obj.get("verdict", {})
    if isinstance(verdict, dict):
        for k in REQUIRED_VERDICT_KEYS:
            if k not in verdict:
                errors.append(f"missing verdict key: {k}")
        if "vulnerable" in verdict and not isinstance(verdict["vulnerable"], bool):
            errors.append(f"verdict.vulnerable should be bool, got {type(verdict['vulnerable']).__name__}")
    else:
        errors.append(f"verdict should be dict, got {type(verdict).__name__}")
    return errors


def parse_verdict(raw_output: str) -> Dict[str, Any]:
    """Parse a model's raw text response into a structured verdict.

    Returns dict with: parsed, errors, vulnerable, confidence.
    """
    obj = extract_json(raw_output)
    if obj is None:
        return {
            "parsed": None,
            "errors": ["no valid JSON verdict found in output"],
            "vulnerable": None,
            "confidence": None,
        }
    schema_errors = validate_schema(obj)
    verdict = obj.get("verdict", {})
    return {
        "parsed": obj,
        "errors": schema_errors,
        "vulnerable": verdict.get("vulnerable") if isinstance(verdict, dict) else None,
```

```
        "confidence": verdict.get("confidence") if isinstance(verdict, dict) else None,
    }


def main() -> None:
    """CLI: render the prompt for a code file (or stdin) and print it."""
    code = open(sys.argv[1]).read() if len(sys.argv) > 1 else sys.stdin.read()
    print(build_prompt(code))


if __name__ == "__main__":
    main()
```

S4 Pairwise Accuracy Example

Fig. S4.1 illustrates how performance on conventional metrics can mask poor *discrimination* ability, that is, the ability to distinguish vulnerable from benign code. Consider three detectors with different approaches:

1) Detector A marks vulnerable code as safe in one pair, and safe code as vulnerable in another.
2) Detector B identifies one pair correctly, but in the second pair it marks the vulnerable code as safe and incorrectly flags the safe code as vulnerable.
3) Detector C marks everything as vulnerable.

A pair is correct only when the vulnerable function is labeled vulnerable and its fixed version is labeled safe.

Detector A

	Vulnerable code	Fixed code	Pair correct?
Pair 1	FLAGGED	FLAGGED	×
Pair 2	SAFE	SAFE	×

Accuracy: 0.50
F1: 0.50
Pairwise accuracy: **0.00**

Detector B

	Vulnerable code	Fixed code	Pair correct?
Pair 1	FLAGGED	SAFE	✓
Pair 2	SAFE	FLAGGED	×

Accuracy: 0.50
F1: 0.50
Pairwise accuracy: **0.50**

Detector C: Flags everything

	Vulnerable code	Fixed code	Pair correct?
Pair 1	FLAGGED	FLAGGED	×
Pair 2	FLAGGED	FLAGGED	×

Accuracy: 0.50
F1: 0.67
Pairwise accuracy: **0.00**

Figure S4.1: A pair is correct only when vulnerable code is flagged and fixed code is not.

One can see how Detectors A and C achieve above-zero accuracy and F1 scores, with Detector C even achieving the best F1, but their pairwise accuracies are 0.

S5 RQ1: Full Metric Breakdown

This section provides supporting analyses for RQ1. Table S5.1 compares the seed and best-pairwise harness across all metrics. Table S5.2 reports how pairwise accuracy correlates with other metrics, and Table S5.3 counts how many harnesses improve over the seed. Overall, pairwise accuracy aligns with accuracy and precision, but not consistently with recall, CWE identification, localization, or Joint.

Table S5.1: Seed and best-pairwise harness performance on TitanVul+CWEval (202 pairs). GLM-5.3 ties at iterations 4 and 6.

Agent + Model	Harness	Pairwise	Accuracy	Precision	Recall	F1	CWE acc	Loc F1	Loc hit	Joint
Codex + Inkling-Small	Seed	6.4	0.520	0.511	0.955	0.665	30.8	33.0	61.7	15.0
	Best pairwise (iter 12)	**27.7**	0.589	0.572	0.695	0.627	18.8	28.5	59.7	9.4
Codex + DeepSeek V4 Flash	Seed	14.4	0.552	0.531	0.896	0.667	42.1	32.0	69.1	22.8
	Best pairwise (iter 11)	**36.1**	0.651	0.660	0.624	0.641	36.0	28.6	51.6	20.9
Claude Code + Opus 4.6	Seed	10.4	0.542	0.524	0.926	0.669	46.2	36.4	68.5	24.4
	Best pairwise (iter 3)	**33.7**	0.653	0.634	0.728	0.677	39.8	39.8	67.3	28.6
Claude Code + GLM-5.3	Seed	19.8	0.572	0.549	0.807	0.653	40.0	32.3	73.0	21.9
	Best pairwise (iter 4)	**31.2**	0.616	0.609	0.648	0.628	40.4	32.6	72.5	29.8
	Best pairwise (iter 6)	**31.2**	0.635	0.641	0.615	0.628	47.7	37.4	71.5	34.1

Table S5.2: Correlation between pairwise accuracy and other metrics. Positive values mean the metrics improve together; negative values indicate a trade-off.

Metric	Inkling-Small	DeepSeek V4 Flash	Opus 4.6	GLM-5.3
Accuracy	$+0.88$	$+0.86$	$+1.00$	$+0.64$
Precision	$+0.92$	$+0.89$	$+0.40$	$+0.76$
Recall	-0.72	-0.21	0.00	-0.77
F1	-0.61	$+0.07$	$+0.40$	-0.47
CWE acc (any-match)	-0.63	-0.33	-0.50	$+0.33$
CWE acc (first CWE)	-0.54	-0.28	-0.10	$+0.80$
Loc precision	-0.36	$+0.19$	$+0.40$	$+0.08$
Loc recall	$+0.02$	-0.19	-0.90	-0.04
Loc F1	-0.26	-0.27	$+0.10$	$+0.29$
Loc hit rate	$+0.20$	-0.43	-0.60	$+0.02$
Joint (any-match CWE)	-0.73	-0.56	-0.80	$+0.44$
Joint	-0.64	-0.54	-0.10	$+0.84$

Table S5.3: Number of harnesses that beat the seed on pairwise accuracy, localization, and Joint. The last two columns show the largest gains over the seed.

	Harnesses beating the seed on				Largest gain over seed	
Agent + Model	Pairwise	+ Loc hit	+ Loc F1	+ Joint	Pairwise	Loc F1
Codex + Inkling-Small	15 / 15	9	2	2	$+21.3$	$+0.9$
Codex + DeepSeek V4 Flash	13 / 13	0	1	4	$+21.8$	$+3.7$
Claude Code + Opus 4.6	4 / 4	1	4	3	$+23.3$	$+4.4$
Claude Code + GLM-5.3	8 / 8	0	6	7	$+11.4$	$+5.1$
Total	40 / 40	10	13	16		

S6 Additional Results

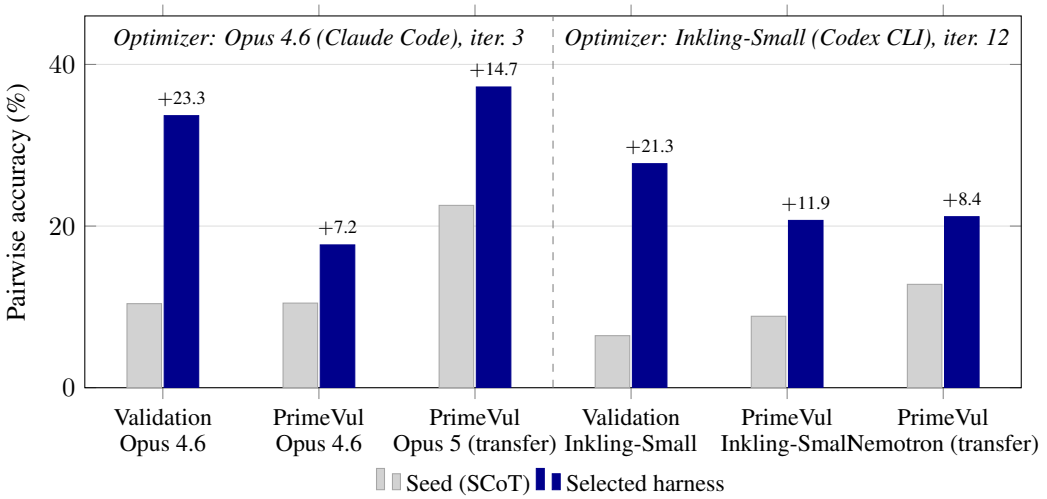


Figure S6.1: Pairwise accuracy of the seed and the harness selected on validation; labels give the gain in points. Transfer bars use a model that took no part in optimization.

S6.1 Expanded per-iteration plots

Figs. S6.2–S6.5 repeat the columns of Fig. 3 at full width, with the value of every point printed. Light yellow bands mark the Pareto frontier on validation and stars mark the best pairwise accuracy of each series, as in Fig. 3.

Due to constrained hosted-model inference token budgets, we ran four refinement iterations with Opus 4.6 and five with GPT-5.4-mini rather than the 15 iterations used for Inkling-Small.

GPT-5.4-mini improved pairwise validation accuracy from 16.83% for the seed harness to 23.27% at iteration 1, a gain of 6.44 points. The validation-selected harness achieved 16.78% on PrimeVul. Because a matching held-out seed evaluation wasn't recorded, we report the absolute held-out score rather than an improvement.

Table S6.1: Reconstructed Inkling-Small token usage per validation sample.

Iter	harness	Completion	Total
0 (seed)	915.9	605.0	1,520.9
1	1,297.9	654.7	1,952.6
2	1,515.9	665.6	2,181.5
3	1,822.4	602.8	2,425.3
4	3,823.3	825.7	4,649.0
5	1,845.7	696.1	2,541.9
6	1,791.1	738.3	2,529.4
7	1,930.1	761.7	2,691.8
8	1,758.1	938.6	2,696.7
9	1,930.1	929.0	2,859.1
10	1,933.1	896.4	2,829.5
11	2,010.1	840.4	2,850.6
12 (selected)	2,105.1	809.2	2,914.4
13	2,387.1	808.0	3,195.1
14	2,295.1	787.5	3,082.6
15	2,786.1	915.2	3,701.3

Means over 404 functions, using the official tokenizer on logged text. Excludes hidden Codex instructions and discarded retries.

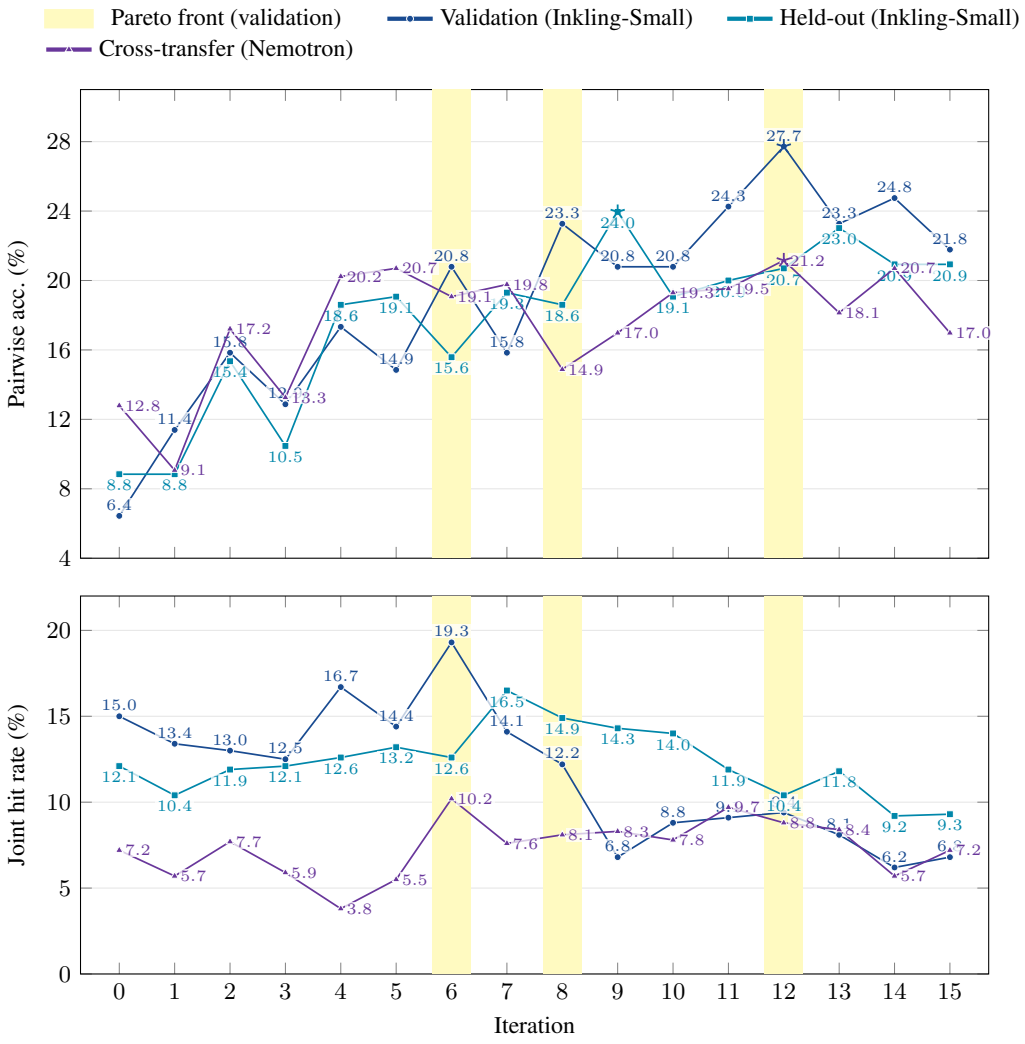


Figure S6.2: Expanded Fig. 3(a): Codex + Inkling-Small.

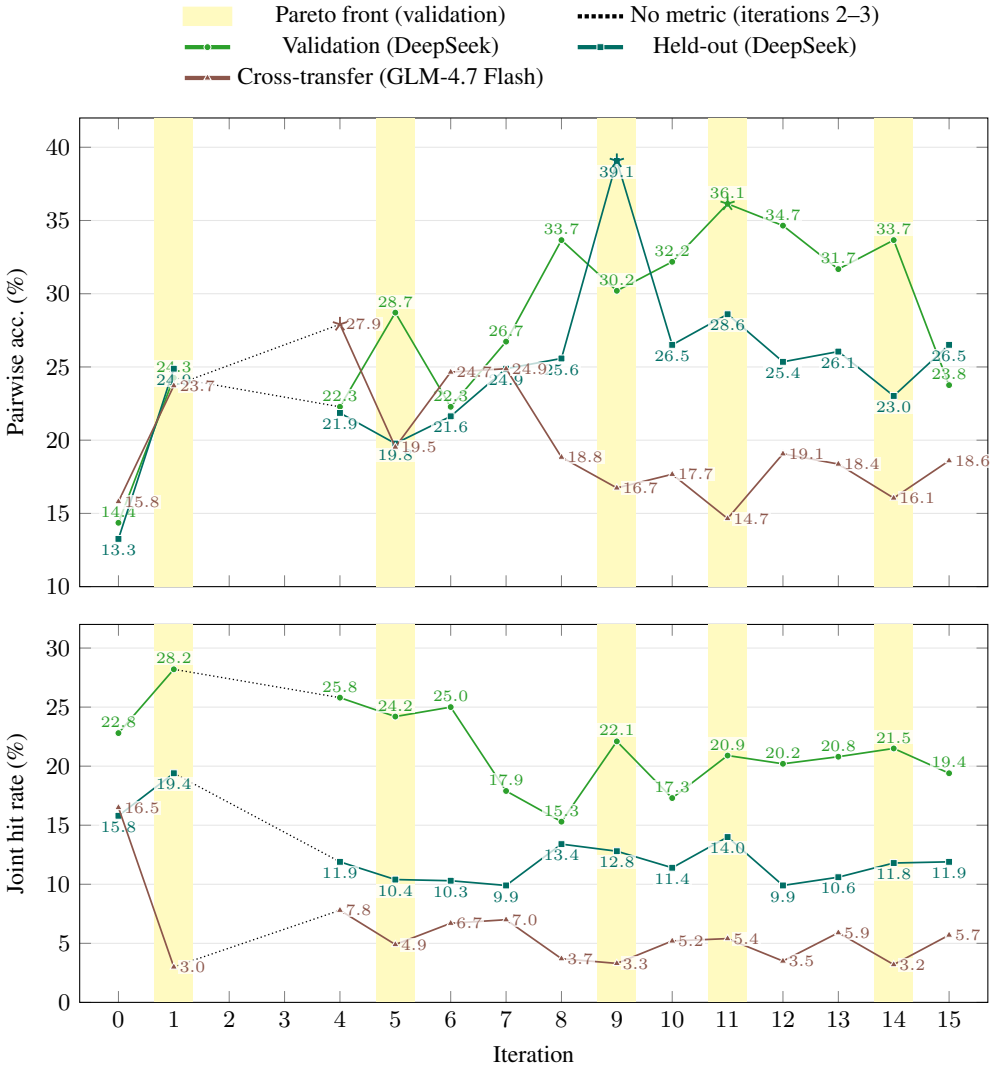


Figure S6.3: Expanded Fig. 3(b): Codex + DeepSeek V4 Flash.

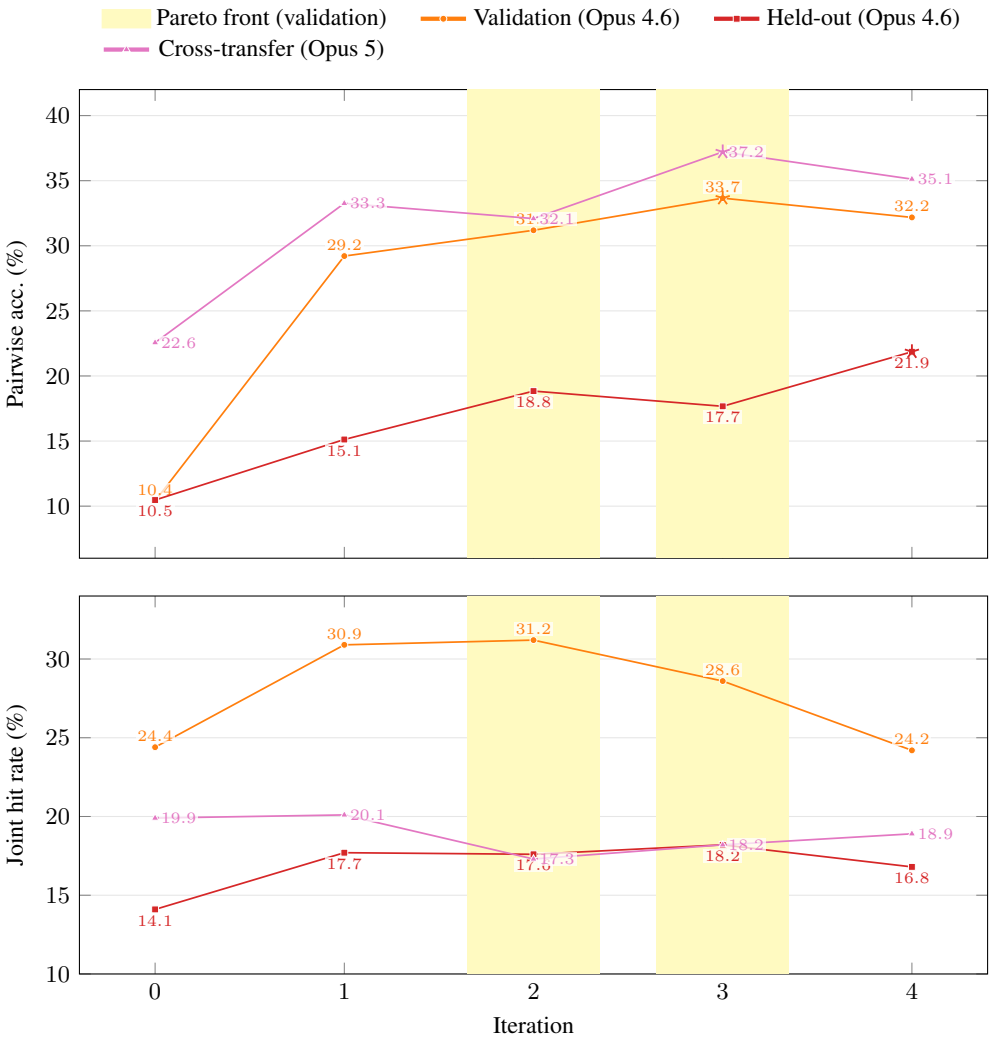


Figure S6.4: Expanded Fig. 3(c): Claude Code + Opus 4.6.

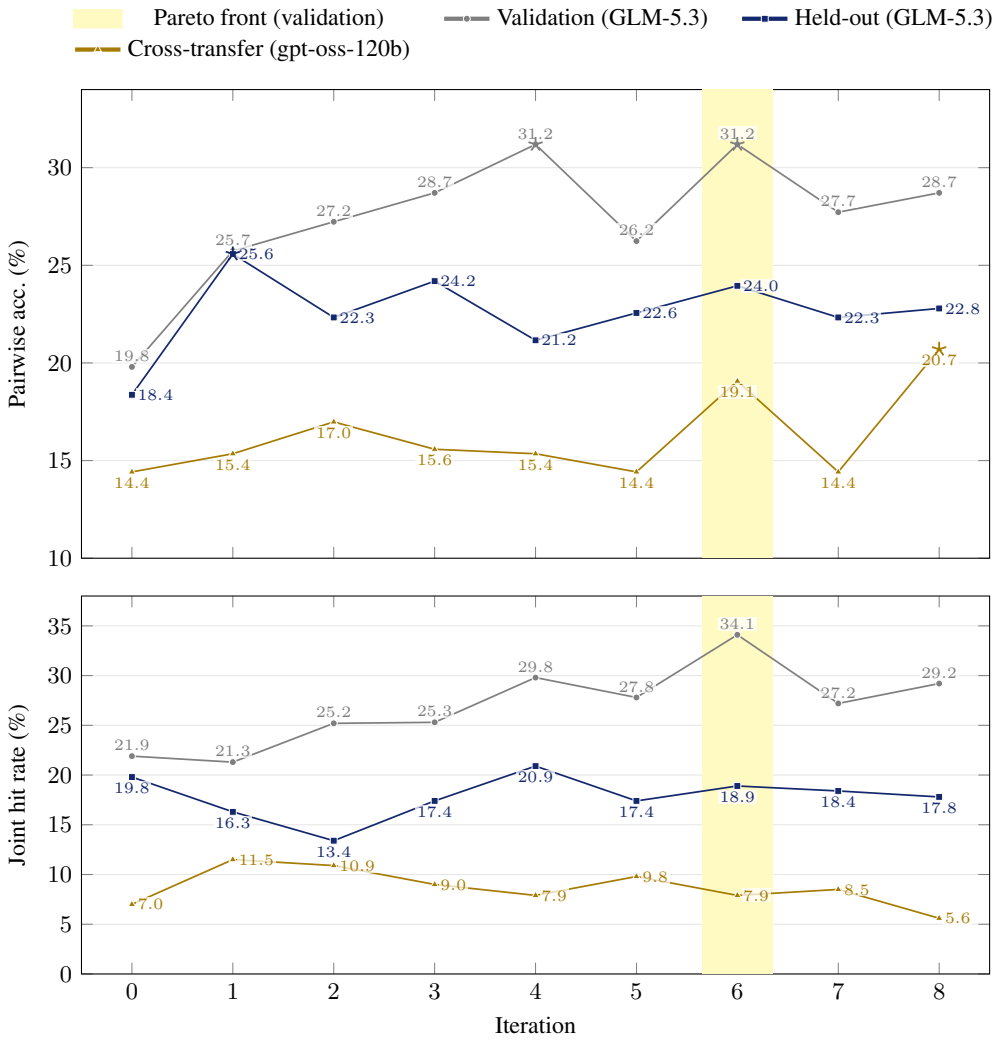


Figure S6.5: Expanded Fig. 3(d): Claude Code + GLM-5.3.

S6.2 WORKED EXAMPLE: SELECTING THE PARETO FRONTIER

We use the Inkling-Small run (Fig. S6.2) to show how MetaVul selects its Pareto frontier. Each of the 16 evaluated harnesses (the seed and iterations 1–15) is a point with two validation objectives, pairwise accuracy and joint hit rate, both maximized (Table S6.2). We say harness A *dominates* harness B if A scores no lower than B on both pairwise accuracy and joint hit rate, and higher than B on at least one of those metrics. The frontier contains those harnesses which are not dominated by any other harness. If two or more harnesses have identical scores on both metrics, we keep only the earliest one.

- Iteration 12 has the highest pairwise accuracy (27.7), so no harness can dominate it.
- Iteration 6 has the highest joint hit rate (19.3), so no harness can dominate it either.
- Iteration 8 (23.3, 12.2) is also undominated: every harness with at least its pairwise accuracy (iterations 11–14) has a lower joint hit rate (6.2–9.4).
- Every other harness is dominated. For example, the seed (6.4, 15.0) is dominated by iteration 6 (20.8, 19.3), and the final iteration 15 (21.8, 6.8) is dominated by iteration 12 (27.7, 9.4).

The frontier is therefore iterations 6, 8, and 12, the yellow bands in Figs. 3 and S6.2. It trades pairwise accuracy against joint hit rate: iteration 12 discriminates best, while iteration 6 names and locates vulnerabilities best.

Table S6.2: Validation objectives for every Inkling-Small harness (%). For dominated harnesses, one dominating harness is listed.

Iter.	Pairwise	Joint	Status
0	6.4	15.0	dominated by 6
1	11.4	13.4	dominated by 6
2	15.8	13.0	dominated by 6
3	12.9	12.5	dominated by 6
4	17.3	16.7	dominated by 6
5	14.8	14.4	dominated by 6
6	20.8	19.3	**frontier**
7	15.8	14.1	dominated by 6
8	23.3	12.2	**frontier**
9	20.8	6.8	dominated by 6
10	20.8	8.8	dominated by 6
11	24.3	9.1	dominated by 12
12	27.7	9.4	**frontier**
13	23.3	8.1	dominated by 8
14	24.8	6.2	dominated by 12
15	21.8	6.8	dominated by 12

S7 GPT-4o Transfer Runs

Table S7.1 lists every harness we evaluated on GPT-4o, the backbone used for the comparison with prior methods in Sec. 6.

Table S7.1: Cross-transfer to GPT-4o on PrimeVul (860 functions, 430 pairs). Harnesses learned with DeepSeek V4 Flash (Codex CLI) are evaluated on GPT-4o, which took no part in optimization. Pairs gives correctly classified pairs out of 430 (samples without a prediction count as incorrect). (sel.) marks the harness selected on validation. Only the iterations shown were evaluated.

Iter	N	Pairwise	Pairs	Acc	F1	Prec	Rec	CWE acc	Loc F1	Loc hit	Parse
Harness learned with DeepSeek V4 Flash (Codex CLI)											
0 (seed)	860	16.51	71/430	51.7	0.543	0.516	0.574	14.6	15.5	27.9	0
7	860	22.09	95/430	53.5	0.593	0.527	0.677	15.1	11.9	19.6	0
8	859	18.37	79/430	54.0	0.388	0.579	0.291	20.0	16.9	26.4	1
10	860	23.49	101/430	54.2	0.495	0.551	0.449	15.5	10.7	17.1	0
11 (sel.)	860	20.70	89/430	53.0	0.500	0.534	0.470	15.8	15.0	22.3	0
12	860	23.26	100/430	54.8	0.485	0.563	0.426	14.2	18.1	25.7	0
14	860	21.86	94/430	54.4	0.453	0.566	0.377	14.8	13.0	19.8	0

S8 AGENT SANDBOXING DETAILS

We provide the sandboxing details for the optimizer and detector agents. This section lists the restrictions summarized in Sec. 3.

S8.1 OPTIMIZER AGENT

Claude Code optimizer.

- **Working directory:** The optimizer starts in its own per-iteration workspace, `workspaces/opt_<session>_iter<N>_optimizer/`.
- **No file writes or edits:** The Write and Edit tools are denied, since headless mode has no one to approve them.
- **Read tool limited to the workspace:** Reading any file outside the workspace requires approval, so it is denied.
- **`find` and `grep` limited to the workspace:** Running these commands on folders outside the workspace is blocked.
- **Shell commands:** Every shell command requires approval, except read-only commands and `python3`.
- **Non-interactive:** The optimizer runs as a single `-p` call with no approval prompts, so every permission request is denied.
- **Fixed system prompt:** Our `OPTIMIZER_SYSTEM` prompt replaces Claude Code's default system prompt.
- **Subscription auth only:** `ANTHROPIC_API_KEY` is removed from the optimizer's environment.
- **Time limit:** Each call is bounded by `runner.timeout_s`.
- **Output:** Only the optimizer's final message is used as the new harness.
- **Validation:** The new harness must keep the SCoT scaffold and the verdict schema; otherwise, the iteration counts as failed.
- **Input:** The optimizer sees only the current workspace's trajectory (harnesses, metrics, history, and labeled predictions). PrimeVul is never copied into the workspace.

Codex optimizer.

- **Working directory:** The optimizer runs in its own per-iteration workspace (`--cd <workspace>`).
- **OS sandbox:** The optimizer runs under the `benchmark_readonly` permission profile, which gives it read-only access to the workspace and to minimal system runtime paths, no other filesystem access, and no write access anywhere.
- **No network:** Network access is disabled in the permission profile.
- **No web search:** Web search is disabled.
- **No approvals:** `approval_policy` is set to `"never"`, so the optimizer cannot escalate its permissions.
- **Isolated setup:** Each call uses a fresh temporary Codex home with only the auth file copied in, so no user config, skills, MCP servers, or memories are loaded.
- **No instruction files:** `AGENTS.md` and other project docs are disabled (`project_doc_max_bytes = 0`).
- **No sub-agents:** Sub-agents are disabled (`multi_agent = false`).
- **Ephemeral session:** Each session is ephemeral (`--ephemeral`), and the Codex rules files are ignored (`--ignore-rules`).
- **Time limit:** Each call is bounded by `runner.timeout_s`.

- **Output:** Only the optimizer's final message is used as the new harness, and it goes through the same SCoT and schema validation.
- **Input:** The optimizer sees only the current workspace's trajectory (harnesses, metrics, history, and labeled predictions). PrimeVul is never copied into the workspace.

S8.2 Detection Agent (Evaluator)

Claude Code detection agent.

- **No tools:** The detector runs with no tools (`--tools ""`), so it cannot read files or run commands.
- **Empty system prompt:** The detector runs with an empty system prompt (`--system-prompt ""`).
- **Working directory:** The detector runs from `/tmp`, so it has no workspace and no repository instructions.
- **Fresh process per function:** Each function is evaluated in a new process, so nothing is remembered between calls.
- **Input:** The detector receives only the rendered harness with one function. It never sees labels, the trajectory, or metrics.
- **Time limit:** Each call is bounded by `runner.timeout_s`.

Codex detection agent.

- **Working directory:** Each function is evaluated in an empty temporary directory, created fresh for that call.
- **OS sandbox:** The detector runs under the same `benchmark_readonly` sandbox, so it is read-only, cannot write, and has no network access.
- **Other restrictions:** Web search is disabled, and the detector has no approvals, sub-agents, instruction files, user config, skills, MCP servers, or memories.
- **Ephemeral session:** Each function gets an ephemeral session and a fresh temporary Codex home.
- **Input:** The detector receives only the rendered harness with one function. It never sees labels, the trajectory, or metrics.
- **Time limit:** Each call is bounded by `runner.timeout_s`.

S8.3 Workspace-Level Restrictions

These restrictions apply to all runs.

- **Separate workspaces:** The optimizer's workspace (suffix `_optimizer`) contains labels and is kept separate from any evaluator copy, which has its labels stripped (`_copy_predictions_no_labels`).
- **Validation data only:** PrimeVul data and predictions are never placed in any workspace.
- **Fixed model weights:** Only the harness text changes between iterations; the model weights stay fixed.

S9 Run Configurations

Per-run settings. In the Inkling-Small and DeepSeek V4 Flash runs, the detector runs in Codex CLI with reasoning effort set to high, served through Tinker and OpenRouter, respectively. In the Opus 4.6 run, it runs in Claude Code through our subscription, and in the GLM-5.3 run, in Claude Code through OpenRouter. In the DeepSeek V4 Flash, Opus 4.6, and GLM-5.3 runs, the optimizer uses the same agent and model as the detector. The Inkling-Small, DeepSeek V4 Flash, and GLM-5.3 runs allow up to 15 refinement iterations with a patience of 4 (GLM-5.3 stopped after iteration 8); each detector call has a 900-second timeout and up to three retries, with 30 concurrent calls in the Codex runs and 60 in the GLM-5.3 run. Opus 4.6 ran four refinement iterations because of subscription limits. The run configurations specify greedy decoding (temperature 0, top-p 1, at most 4,096 output tokens), but the Claude CLI does not forward these settings to the model. The Opus 4.6 and Opus 5 seed baselines come from standalone runs of the seed harness rather than from the discovery session.

Opus 4.6 ran through our Claude subscription rather than per-token API billing. Subscription limits motivated a maximum of five iterations; the recorded run contains the seed and four refinement iterations. OpenRouter and Tinker runs instead used hosted inference through the local LiteLLM proxy. The OpenRouter configuration pins DeepSeek V4 Flash to FP8 providers and Nemotron to BF16 providers; the Tinker Nemotron run uses the explicit BF16 model identifier. These are separate serving routes, not interchangeable pricing records. We routed Claude Code through a local LiteLLM proxy, which translated its Anthropic Messages API requests into OpenRouter chat completion requests. Each model was registered in the proxy configuration with authentication supplied through an API key (environment variable). Claude Code's auxiliary model aliases were accordingly pinned to the same target model. For GLM-5.3 experimentation, both the optimizer and evaluator used this route. For the GLM-5.3 run, each evaluation attempt had a 900-second timeout and up to three retries. The Claude CLI output cap was 32,000 tokens. Although the saved configuration lists temperature 0, top-p 1 and a 4,096-token limit, the Claude CLI model client does not forward these decoding fields; we therefore do not report them as effective sampling settings. Current provider prices are listed in Supp. S12; they are not measurements of total experiment cost. Run lengths depended on inference budgets and stopping criteria. GLM-5.3 used maximum of 15 refinement iterations (T=15) and patience of 4 iterations without improvement (K=4), stopping after iteration 8.

S10 CONTEXT BUDGET

Each base LLM runs inside the designated agent's CLI (either Codex or Claude Code), which manages the model's context. The CLI exposes file tools (listing, search, and reading) and runs them, appends the output to the conversation, and caps how much a single call returns. When the conversation approaches the model's context window, the CLI compacts it, and replaces the earlier turns with a summary.

The optimizer's history is stored as files in a read-only workspace. For each session, we take the workspace the optimizer was given on its last call and count its tokens with the `o200k_base` tokenizer. The saved workspaces range from 1.04M to 4.21M tokens, larger than any context window we use. The optimizer therefore never loads the history in full: it browses the workspace with these tools, and only the files it opens enter its context.

In Table S10.1, we show the detection call stats. Each detection call, unlike what the optimizer has, sends one harness and one function. For each code sample, the evaluation pipeline calls the harness's build function, which inserts the code into the harness's template. The detector then sends the resulting filled-in template to the LLM. The largest detection call was about 77K tokens, from a harness learned in the DeepSeek run. It fit easily within the windows of both models that ran it: 1.31M tokens for DeepSeek V4 Flash and 200K for GLM-4.7 Flash. Nemotron had the smallest window, which we set to 65.5K, and its largest call was only 22.5K. No detection call exceeded its model's window, so no harness or function was ever cut off.

Table S10.1: Context window and the size of one detection call (the harness plus one PrimeVul function), per model.

Model	Role	Agent	Context window	Per detection call (median / max)
Claude Opus 4.6	optimizer, detector	Claude Code	200K	3.4K / 23.7K
Claude Opus 5	detector (transfer)	Claude Code	200K	3.5K / 23.7K
GLM-5.3	optimizer, detector	Claude Code	200K[a]	5.8K / 27.8K
gpt-oss-120b	detector (transfer)	Claude Code	131K	5.8K / 27.8K
Inkling-Small	optimizer, detector	Codex CLI	256K	2.4K / 44.4K
Nemotron 3.5 Lightning	detector (transfer)	Codex CLI	65.5K[b]	2.5K / 22.5K
DeepSeek V4 Flash	optimizer, detector	Codex CLI	1.31M	5.7K / 76.9K
GLM-4.7 Flash	detector (transfer)	Codex CLI	200K	6.9K / 76.9K
GPT-4o	detector (transfer)	Codex CLI	128K	5.7K / 53.7K

[a] The model accepts 1.31M tokens; Claude Code runs it with the 200K window of the Claude model it is mapped to. [b] Set for Codex; the model accepts up to 1M. Token counts are estimated at 4 characters per token.

S11 OPTIMIZER-INFORMATION ABLATION

In the following experiment, we are testing ablation on the validation data only. To do so, we run the discovery loop with 5 levels of information: starting with the full workspace, to having only the harness as a signal (Table S11.1). We use `gpt-5.6-luna` for this, due to cost. We use medium reasoning effort for both the detection and optimizer agents. We run 5 iterations for each of these 5 levels. This means the agent has 5 iterations to improve the harness, having only access to that "level" of information as the signal. All levels start from the same seed harness, and only differ by what the level's signals contain. The *Optimizer workspace* column reveals what the workspace has access to. For `no-labels`, fields whose position reveals the label were also removed, and sample identifiers were replaced by opaque hashes.

Overall, our findings are as follows. All levels improved significantly over the seed (from 9.9 to between 19.8 and 23.3 pairwise accuracy, exact McNemar $p \leq 0.005$), and no level differed significantly from another (all $p \geq 0.33$). We consider this ablation inconclusive: it uses a single run per level, a single optimizer configuration (`gpt-5.6-luna`, an efficiency-oriented model, at medium reasoning effort) that differs from those in our main experiments, and validation data only.

Level	Optimizer workspace	It. 0	It. 1	It. 2	It. 3	It. 4	It. 5	Best	Mean
`full`	harnesses, metrics, history, predictions with labels	9.90	19.80	14.36	18.81	13.37	17.33	19.80	16.73
`no-labels`	same, labels and label-revealing fields removed	9.90	16.83	19.80	19.80	20.30	17.82	20.30	18.91
`no-predictions`	harnesses, metrics, history	9.90	21.78	16.83	19.31	19.31	19.31	21.78	19.31
`no-history`	latest metrics only	9.90	16.83	19.31	21.29	23.27	19.80	23.27	20.10
`harness-only`	current harness only	9.90	16.34	20.79	19.31	15.84	21.29	21.29	18.71

Table S11.1: Optimizer-information ablation on the validation set (202 pairs, 404 functions), **pairwise accuracy** (%). All levels start from the same seed harness (It. 0). Mean is over iterations 1–5.

S12 HOSTED INFERENCE PRICES

OpenRouter entries are based on the model card's profile headline rates on `openrouter.ai`. Input/output price always varies by routing, precision, caching, and discounts, but these were the prices when we ran our experimentation. Tinker prices include any listed discount. We link the model names to the sources. We used Opus through the subscription (`claude -p`).

Model	Service	Input	Output
GLM-5.3	OpenRouter	0.5614	1.764
DeepSeek V4 Flash 0731	OpenRouter	0.022	0.32
GLM-4.7 Flash	OpenRouter	0.06	0.40
GPT-OSS-120B	OpenRouter	0.03	0.17
GPT-4o	OpenRouter	2.50	10.00
Inkling-Small (64K)	Tinker	0.58	1.44
Nemotron 3.5 Lightning BF16 (64K)	Tinker	0.195	0.495
Claude Opus 4.6	Anthropic subscription[a]	5.00	25.00
Claude Opus 5	Anthropic subscription[a]	5.00	25.00

[a] Run through a subscription; prices shown are Anthropic's API list prices, for reference.

Table S12.1: Listed inference prices in USD per million tokens.